SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: MLA0303
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO2: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10 **Total Marks: 50**

Annexure A
Application - Based Questions

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**

2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**

3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**

4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**

5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

(5 Marks)

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance**.** **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

I ***A.Sai Rohit (192472144)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *A.Sai Rohit*

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality.

Introduction

Ride-sharing platforms such as Uber and Ola operate in a highly dynamic environment where thousands of drivers and passengers interact in real time across a city. Reinforcement Learning (RL) is well suited to this problem because the platform (agent) must continuously make sequential decisions — which driver to assign to which passenger — under uncertainty, and the outcome of one decision (e.g., sending a driver far away) affects future states of the system. Unlike a purely greedy assignment rule, RL naturally accounts for the long-term, cumulative effect of each matching decision on the overall health of the marketplace. The goal of the RL framework is to learn a matching policy that minimizes passenger waiting time, minimizes driver idle time, and maximizes overall service quality and revenue, while keeping the system stable during sudden demand surges such as concerts, festivals, or bad weather.

1. State Space (S)

The state must capture all information relevant to making a good matching decision at a given time step. A representative state vector can include:

- Current GPS location of every available driver and every waiting passenger.
- Number of idle drivers and number of pending ride requests in each geographic zone.
- Time of day and day of the week (captures demand patterns such as rush hour or weekends).
- Traffic congestion level and estimated travel time between zones.
- Driver-specific attributes such as vehicle type, rating, and fuel/battery level.
- Historical demand-supply imbalance in nearby zones (to anticipate future requests).
- Weather conditions and local events that may temporarily spike demand in a zone.
- Surge-pricing multiplier currently active in each zone.

2. Action Space (A)

The action space consists of the decisions the agent can take at each state:

- Assign a specific idle driver to a specific waiting passenger.
- Instruct an idle driver to reposition to a high-demand zone in anticipation of future requests.
- Hold/delay a match momentarily if a better match is likely to appear shortly (batching requests).

- Reject or defer a request when no driver is reasonably close (with surge-pricing trigger as an alternative action).
- Pool multiple passengers travelling along a similar route into a single shared ride.

3. Reward Function (R)

The reward function must be engineered so that maximizing cumulative reward corresponds to the platform's real objectives. A composite reward can be defined as:

- A negative reward proportional to passenger waiting time (the longer the wait, the larger the penalty), which pushes the agent to minimize waiting time.
- A positive reward for successfully completing a trip, scaled by fare or trip value.
- A small negative reward for driver idle time or empty repositioning distance, encouraging efficient driver utilization.
- A positive reward component tied to customer satisfaction ratings after the trip.
- A penalty for cancellations caused by poor matches (e.g., driver too far away).
- A bonus reward for successfully balancing supply and demand across zones (reducing city-wide imbalance).

4. Algorithmic Approach

Because the state and action spaces are extremely large (thousands of drivers and riders across a city, updating every few seconds), tabular methods such as plain Q-Learning are impractical. Instead, the problem is typically modelled as a large-scale Markov Decision Process (MDP) and solved using Deep Reinforcement Learning approaches such as Deep Q-Networks (DQN), which approximate the Q-value using a neural network, or multi-agent RL frameworks where each driver or dispatch zone is treated as a cooperating agent. Value-based methods estimate the long-term value of assigning a driver to a rider from a particular zone, while policy-gradient methods such as PPO directly learn the probability of taking each dispatch action. Dynamic Programming concepts such as the Bellman Optimality Equation underpin the theoretical foundation, even though in practice approximate/dynamic programming (value iteration on a discretized zone-level MDP) is used for scalability.

5. Worked Illustration

Consider a city divided into 50 zones. During evening rush hour, zones near residential areas show a surge in demand while zones near office parks have many idle drivers finishing their trips. Instead of only matching passengers to the nearest available driver (a greedy/myopic approach), the RL agent — having learned from historical patterns — proactively repositions idle drivers from office-park zones toward residential zones a few minutes before the predictable demand surge begins. This anticipatory action, driven by the learned value function,

reduces the average wait time during the rush period far more than a reactive, greedy dispatch policy would.

6. Learning Process and Outcome

Using an RL algorithm such as Deep Q-Networks (DQN) or Proximal Policy Optimization (PPO), the agent interacts with the simulated or live environment, observes the state, selects a matching action, receives the reward, and updates its policy to favor actions that historically reduced waiting time and increased successful, high-rated trips. Over many episodes, the agent learns an optimal policy $\pi^*(s)$ that dynamically balances immediate matching against anticipatory repositioning, resulting in reduced average wait times, better driver utilization, and improved overall service quality across the city.

7. Conclusion

By formally defining the state space, action space, and a carefully engineered multi-component reward function, and by applying scalable Deep RL and dynamic-programming-based methods, a ride-sharing company can build an intelligent dispatch system that continually improves itself from operational data, reducing rider waiting time, improving driver earnings consistency, and enhancing the overall efficiency and profitability of the platform.

2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

Introduction

In an e-learning recommendation system, the RL agent must decide which study material (video, quiz, article, or practice set) to recommend to a learner next. The central challenge is the exploration–exploitation trade-off: should the system exploit content it already knows the learner enjoys and performs well with, or should it explore new/untested content that might be even more effective for that learner's long-term learning outcomes? Because every learner has a different background, pace, and learning style, this trade-off must be resolved individually for each learner rather than through a single one-size-fits-all rule.

1. State, Action, and Reward (Context)

The state typically represents the learner's profile: topics already mastered, recent quiz performance, time spent per content type, engagement level, and preferred learning style (visual, textual, or interactive). The action is the specific study material recommended next (a particular video, article, quiz, or practice set). The reward is derived from the learning outcome that follows the recommendation — for example, an increase in quiz score, time spent actively engaged, successful completion of the material, or a positive explicit rating from the learner.

2. Exploitation

Exploitation means the agent recommends the content type or topic that has historically produced the highest reward (e.g., high quiz scores, high engagement, high completion rate) for a learner with a similar profile. This maximizes short-term satisfaction and learning efficiency, but if used exclusively, the system will keep recommending the same narrow set of materials and never discover potentially better content, leading to a suboptimal long-run policy — a phenomenon known as getting stuck in a local optimum.

3. Exploration

Exploration means the agent occasionally recommends new, less-tried, or diverse study material — content the learner hasn't engaged with before — purely to gather information about its effectiveness. This helps the system discover new content-learner matches (for example, a learner who has only used video lectures might actually learn better from interactive simulations, but the system would never find this out without exploring).

4. Balancing Strategies

Several standard RL strategies help balance the two:

- Epsilon-Greedy: With probability ε the agent recommends a random/new material (exploration); with probability $(1-\varepsilon)$ it recommends the best-known material (exploitation). ε is typically decayed over time as the system learns more about the learner.
- Upper Confidence Bound (UCB): The agent favors content with high estimated reward but also gives a bonus to content that has been tried less often, since its true value is more uncertain.
- Thompson Sampling: The agent maintains a probability distribution over the expected effectiveness of each content type and samples from it, naturally balancing exploration and exploitation based on uncertainty.
- Contextual Bandits/Contextual RL: Exploration is guided by the learner's context (prior knowledge, learning pace) so that exploration is not random but targeted toward plausible useful content.
- Softmax (Boltzmann) exploration: content is chosen probabilistically in proportion to its estimated reward, so promising but under-tried content is still occasionally selected rather than fully random exploration.

5. Worked Illustration

Suppose a learner has consistently scored well on video-based lessons for a topic and the system, purely exploiting this pattern, keeps recommending more videos. Using an epsilon-greedy strategy with $\varepsilon = 0.1$, roughly 10% of the time the platform instead recommends an interactive coding exercise on the same topic. If the learner engages deeply and scores even

higher on the interactive exercise, the system updates its estimate of that content type's value upward, and over subsequent sessions it begins recommending interactive exercises more frequently — effectively discovering a better-matched learning modality that pure exploitation would have missed.

6. Outcome

By carefully balancing exploration and exploitation, the e-learning platform continues to exploit content that is known to work well for a learner while periodically exploring new material to discover even better personalized learning paths, ultimately maximizing long-term learning outcomes and engagement rather than short-term convenience. As more data accumulates for a given learner, the system gradually shifts weight from exploration toward exploitation, reflecting growing confidence in its personalized model of that learner.

3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time.

Introduction

A robotic vacuum cleaner operates as an RL agent in a physical room environment. Its objective is to maximize the cleaned area per unit of battery/time while avoiding collisions with obstacles such as furniture, walls, pets, and stairs (drop-offs). The room can be modelled as a Markov Decision Process where each grid cell is a possible state, and the robot must learn a control strategy purely from its own trial-and-error interaction with the environment (bump sensors, cliff sensors, and dirt sensors).

1. State, Action and Reward

The state can be represented as the robot's current position and orientation on a grid map of the room, the cleanliness status of each grid cell (cleaned or not), sensor readings (proximity/ultrasonic/infrared for obstacle distance), and remaining battery level. The action space includes move forward, turn left, turn right, move backward, and stop/dock for charging. The reward function gives a positive reward for cleaning a new (previously dirty) cell, a large negative reward for colliding with an obstacle or falling off an edge, a small negative reward (step cost) for every time step to encourage efficiency, and a positive reward for successfully returning to the charging dock when battery is low.

2. Policy (π)

The policy $\pi(a|s)$ is the strategy the robot follows — a mapping from the state it perceives to the action it should take. Initially the policy may be close to random, causing inefficient,

repetitive movement and frequent collisions. Through RL training (e.g., Q-Learning or Deep Q-Networks), the robot updates its policy based on the rewards it receives, gradually learning to prefer actions that lead to covering more uncleaned area while steering away from obstacles. The policy can be deterministic (always choosing the single best action in a state) or stochastic (choosing among actions according to a probability distribution, which is useful for maintaining some exploration during early training).

3. Value Function and Its Role

- The value function V(s) estimates the expected cumulative future reward the robot can obtain starting from a given state and following its current policy thereafter. The action-value function Q(s,a) estimates the expected return of taking a specific action in a specific state and then following the policy. These functions act as a long-term 'quality score' for states and actions, rather than only judging the immediate reward.
- As training progresses, the value function is updated using the Bellman equation, which propagates information about future rewards backward to earlier states. For example, a cell located directly in front of a wall corner might have a low immediate risk but the value function will learn that approaching it too fast often leads to collisions later, so it assigns that state a lower value, discouraging the robot from repeating the mistake.

4. Policy Evaluation and Policy Improvement (Dynamic Programming Link)

Formally, this improvement process follows the two alternating steps of Generalized Policy Iteration studied under Dynamic Programming: (a) Policy Evaluation, where the value function V(s) is computed/updated for the current policy using the Bellman Expectation Equation, and (b) Policy Improvement, where the policy is updated to act greedily with respect to the newly evaluated value function, i.e., $\pi'(s) = \text{argmax}_a Q(s,a)$. Repeating these two steps is guaranteed, under standard MDP assumptions, to converge toward the optimal policy π^* and optimal value function $V^*(s)$, which is the theoretical justification for why the robot's cleaning behaviour keeps improving with more experience.

5. Worked Illustration

Early in training, the vacuum might repeatedly bump into a table leg location because it has not yet learned that the cell adjacent to it is risky. After several collisions (each producing a large negative reward), the Q-value for 'move forward' in that particular state decreases sharply relative to 'turn left' or 'turn right'. The policy, updated greedily with respect to Q, then avoids that action in that state in future episodes, and this improved behaviour generalizes to structurally similar states elsewhere in the room (e.g., other table-leg-like corners) once a function-approximation method such as a neural network is used instead of a lookup table.

6. Improvement Over Time

As more cleaning episodes are run (either in simulation or in the real room), the estimated value function converges closer to the true expected return, and the policy is repeatedly improved by acting greedily (or near-greedily) with respect to this increasingly accurate value function — a cycle known as policy iteration/generalized policy iteration. Over time this produces smoother, more systematic coverage paths (e.g., boustrophedon/zig-zag patterns), fewer collisions, better battery management, and higher overall cleaning efficiency.

7. Conclusion

The combination of a well-designed state-action-reward structure with the iterative refinement of the value function and policy allows a robotic vacuum cleaner to transform from an initially clumsy, collision-prone agent into an efficient cleaner that covers the maximum possible area in the minimum time while safely avoiding obstacles, illustrating the direct practical value of dynamic-programming-based RL concepts in embedded robotics.

4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

Introduction

In an RL-based cryptocurrency trading agent, the state typically includes price history, technical indicators (moving averages, RSI, volume), and portfolio holdings, while the action space is {Buy, Sell, Hold}. The reward function is the most critical — and most difficult — component to design correctly, because it directly shapes what behavior the agent will learn to optimize. Unlike supervised learning where a labelled 'correct answer' exists for each state, RL trading agents learn purely from the numeric feedback of the reward signal, so any flaw in the reward design directly becomes a flaw in the trading strategy the agent ultimately learns.

1. State and Action Context

A typical state vector includes recent closing prices, trading volume, volatility measures, technical indicators such as the Relative Strength Index (RSI) and Moving Average Convergence Divergence (MACD), and the agent's current portfolio composition (cash balance and coin holdings). The action space {Buy, Sell, Hold} may be further parameterized with a trade size (what fraction of the portfolio to trade), which increases the complexity of both the action space and the reward attribution problem.

2. Key Challenges in Reward Design

Several challenges arise:

- Sparse and delayed rewards: Profit from a trade may only be realized after holding a position for a long period, making it hard for the agent to associate early actions with eventual outcomes.
- Reward scale and volatility: Cryptocurrency prices are extremely volatile, so a raw profit/loss reward can have huge variance, destabilizing learning.
- Risk-blind rewards: A reward based purely on profit ignores risk; an agent may learn extremely risky strategies (large leveraged positions) that occasionally produce big profit spikes but are catastrophic on average.
- Overfitting to short-term price noise: If the reward is given at every tick based on unrealized profit, the agent may learn to react to random market noise rather than genuine trends.
- Transaction costs and slippage: If the reward function ignores trading fees and slippage, the agent may learn a policy that trades excessively (e.g., buying and selling every step) which looks profitable in simulation but loses money in real markets.
- Reward hacking: The agent may find unintended shortcuts to gain reward, such as exploiting a bug or a specific quirk of the reward formula, rather than actually trading well.
- Non-stationarity of markets: The statistical properties of price movement change over time (bull markets, bear markets, regulatory shocks), so a reward function tuned to one regime may implicitly encode assumptions that no longer hold in another.

3. Effects of Poor Reward Design

If these challenges are not addressed, poor reward design can severely affect performance:

- The agent may become overly aggressive or risk-seeking, chasing short-term reward spikes and suffering large drawdowns.
- The agent may learn a degenerate policy, such as always holding (avoiding risk entirely) if penalties for losses are too harsh relative to rewards for gains.
- Excessive trading frequency can erode profits through cumulative transaction costs if fees are not penalized.
- The learned policy may perform well in backtesting/simulation but fail in live markets (a generalization gap) if the reward function was tuned to historical data quirks.
- The agent may develop a strong bias toward a single asset or direction (always buying) if the training data happened to be dominated by an upward trend, causing it to fail badly when the market reverses.

4. Worked Illustration

Suppose the reward is defined simply as the unrealized profit at every time step. During a strong but short-lived rally, the agent receives large positive rewards for holding an increasingly leveraged long position. When the rally reverses sharply (a common pattern in crypto markets), the same unrealized-profit-based reward turns strongly negative, but by then the position size is large and the resulting loss is severe. A better-designed reward — for example, one based on the realized Sharpe ratio over a rolling window, with an explicit penalty for position size relative to volatility — would have discouraged the agent from building such a large position in the first place.

5. Good Practices

To mitigate these issues, practitioners typically use risk-adjusted reward metrics such as the Sharpe ratio or Sortino ratio instead of raw profit, include explicit transaction-cost and slippage penalties, use reward shaping/normalization to control variance, cap position sizes to bound downside risk, and validate the policy across multiple, non-overlapping market regimes (bull, bear, sideways) to avoid overfitting to a single historical pattern.

6. Conclusion

Reward function design is arguably the single most important and most error-prone step in building an RL-based trading system; a well-designed, risk-aware reward function is essential to obtain a policy that is not just profitable in backtests but also robust, stable, and safe to deploy with real capital.

5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

Introduction

A smart grid must balance electricity supply and demand in real time, incorporating renewable sources (solar, wind), storage (batteries), and fluctuating consumer demand. An RL agent can act as the grid controller, learning to distribute and store electricity optimally to minimize cost, reduce waste, and maintain reliability. This is a naturally sequential decision-making problem: a storage or curtailment decision made now directly affects the options and costs available at the next time step, which is exactly the type of problem Dynamic Programming and RL are designed to solve.

1. State Space (S)

The state should represent the current condition of the grid:

- Current electricity demand across different zones/substations.

- Current generation output from each source (solar irradiance-based output, wind output, conventional generators).
- Battery/storage charge level at each storage unit.
- Grid frequency and voltage stability indicators.
- Electricity market price at the current time step (if trading power is possible).
- Weather forecast data relevant to renewable generation (short-term forecast).
- Time of day/season, since demand patterns and renewable availability both vary predictably with time.

2. Action Space (A)

The controller's possible actions include:

- Allocate a certain amount of power from each generation source to each demand zone.
- Charge or discharge battery storage units by a certain amount.
- Buy or sell electricity from/to the external grid/market.
- Curtail (reduce) renewable generation if there is oversupply, or shed non-critical load if there is undersupply.
- Activate demand-response programs, incentivizing consumers to shift non-critical usage to off-peak periods.

3. Reward Function (R)

The reward mechanism should encourage cost-efficient, stable, and sustainable operation:

- A positive reward for successfully meeting demand without shortage (avoiding blackouts).
- A negative reward proportional to the cost of electricity purchased from external sources or generated from expensive/non-renewable sources.
- A positive reward for maximizing the use of renewable energy over fossil-fuel-based generation.
- A penalty for large deviations in grid frequency/voltage (instability).
- A penalty for excessive battery cycling (to preserve battery lifespan) and for curtailing renewable energy unnecessarily.

4. Algorithmic Approach

Because the action space (how much power to allocate, charge, or discharge) is continuous, algorithms designed for continuous control are preferred over simple tabular Q-Learning. Deep Deterministic Policy Gradient (DDPG) and Soft Actor-Critic (SAC) are commonly used because they can output real-valued actions (e.g., 'discharge battery by 12.5 kWh') rather than

a discrete menu of choices. The underlying theory still traces back to Dynamic Programming: the Bellman equation is used within the actor-critic architecture to evaluate how good a given power-allocation decision is expected to be in the long run, not just at the current instant.

5. Worked Illustration

Consider a grid segment with a solar farm and a battery bank. During midday, solar output exceeds current demand. A naive rule-based controller might simply curtail the excess solar energy (wasting it). The RL agent, having learned the value of stored energy for the evening peak, instead directs the surplus into the battery. In the evening, when demand rises and solar output falls to zero, the agent discharges the pre-charged battery instead of purchasing expensive peak-hour electricity from the external grid, reducing both cost and reliance on non-renewable backup generation.

6. Learning and Outcome

Using an RL algorithm suited for continuous action spaces, such as Deep Deterministic Policy Gradient (DDPG) or Soft Actor-Critic (SAC), the agent learns, through repeated interaction with a simulated grid environment, a policy that dynamically allocates generation and storage resources. Over time it learns to anticipate demand peaks (e.g., evening residential peak) and pre-charge batteries using cheap renewable energy earlier in the day, resulting in lower operating cost, reduced carbon footprint, and a more stable, resilient electricity distribution system.

7. Conclusion

By explicitly modelling the grid as an MDP with a well-designed state, action, and reward structure, and applying continuous-control RL algorithms grounded in dynamic programming principles, a smart grid can achieve significantly better cost efficiency, higher renewable energy utilization, and improved reliability compared to static, rule-based control strategies.

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes.

Introduction

In a healthcare monitoring system, an RL agent can support clinicians by recommending dosage adjustments for a patient's medication (for example, insulin dosing for diabetes management) based on continuously monitored physiological signals. Because patient safety is paramount, such systems are typically developed and validated under strict clinical oversight, often as a decision-support tool rather than a fully autonomous controller. The sequential nature of dosing

— where today's dose affects tomorrow's physiological state — makes this a textbook example of a Markov Decision Process suitable for RL and dynamic-programming-based analysis.

1. State Space (S)

The state should capture the patient's current physiological and clinical condition:

- Current vital signs (blood glucose level, heart rate, blood pressure, oxygen saturation).
- Recent trend of these vitals over the past few time steps (e.g., rising or falling glucose).
- Patient-specific attributes such as weight, age, and known sensitivities/allergies.
- Time since the last dose and current dosage level.
- Activity level or food intake information reported by the patient/wearable device.

2. Action Space (A)

The possible actions represent dosage decisions:

- Increase the medication dosage by a defined increment.
- Decrease the dosage by a defined increment.
- Maintain the current dosage.
- Trigger an alert to a clinician if the patient's state is outside safe operating bounds (a safety fallback action).

3. Reward Function (R)

The reward function must strongly prioritize patient safety alongside effectiveness:

- A positive reward when the patient's vital signs (e.g., blood glucose) stay within the target healthy range.
- A large negative reward for dangerous events, such as hypoglycemia/hyperglycemia or any adverse reaction, with the penalty scaled to the severity of deviation.
- A small negative reward for large or frequent dosage changes, to encourage smooth, clinically stable adjustments rather than erratic dosing.
- A positive reward for maintaining long-term stability across many time steps (rewarding consistency, not just single-step correctness).

4. Safety Constraints and Policy Design

Unlike many other RL applications, a healthcare dosing agent must operate under hard safety constraints, not just soft reward-based preferences — for example, the action space itself may be constrained so that the dosage can never exceed a clinically approved maximum, regardless of what the learned policy would otherwise prefer. Techniques from Safe Reinforcement Learning and Constrained MDPs are used to guarantee that, even during exploration/training,

the agent never takes an action that could push a physiological variable into a dangerous zone. In practice, the policy is usually trained and validated offline on historical patient data or a calibrated physiological simulator before being used, in a supervised capacity, to support (not replace) a clinician's decision.

5. Worked Illustration

Consider a diabetic patient whose blood glucose tends to rise sharply after a particular meal pattern reported through a food-logging app. Initially, a fixed-dose regimen might under-correct after such meals, leading to a recurring post-meal hyperglycemia episode (a state associated with strongly negative reward). As the RL agent accumulates experience, its value function learns to associate this specific state pattern (meal type plus current glucose trend) with the need for a proactively higher insulin dose slightly earlier than the fixed schedule would suggest, thereby keeping glucose within the target range more consistently.

6. Learning and Outcome

Using constrained or safe RL techniques, the agent is typically trained first on historical patient data or a simulated physiological model before any real deployment, ensuring the learned policy respects hard safety constraints. As the agent accumulates experience, its value function learns to anticipate how a given dosage today affects the patient's condition several hours later, allowing it to make proactive rather than purely reactive adjustments. Over time, this leads to tighter control of the target physiological variable, fewer dangerous excursions, and dosage plans that are personalized to each patient's unique response pattern, ultimately improving health outcomes.

7. Conclusion

When designed with rigorous safety constraints and a carefully balanced reward function, an RL-based dosage-adjustment system can complement clinical expertise by continuously personalizing treatment to each patient's unique and evolving physiological response, ultimately supporting safer, more consistent, and more effective long-term disease management.

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization.

Introduction

A drone delivery system uses RL to navigate from a depot to a customer's location while avoiding obstacles, no-fly zones, and adverse weather, all while minimizing battery consumption and delivery time. Route optimization directly depends on how the agent balances exploration (trying new routes) and exploitation (using proven, efficient routes). The three-dimensional nature of drone flight (including altitude choice) makes the route-search space

considerably larger than typical ground-vehicle routing, further increasing the importance of an efficient exploration strategy.

1. State, Action, and Reward (Context)

The state includes the drone's GPS position, altitude, battery level, wind conditions, and the relative position of the destination and known obstacles. The action space includes movement directions (e.g., forward, turn, ascend, descend) or, at a higher level, selection of the next waypoint. The reward function typically gives a positive reward for progress toward the destination and successful delivery, and negative rewards for collisions, entering restricted airspace, or excessive battery usage.

2. Role of Exploitation

Exploitation means the drone follows the route it currently believes is best — for instance, a previously flown path that was short, safe, and fuel-efficient. This ensures reliable, predictable deliveries and avoids wasting battery on risky detours. However, always exploiting the same known route prevents the system from discovering shorter paths that may exist due to changing conditions (e.g., a new gap in a no-fly zone or better wind patterns at a different altitude).

3. Role of Exploration

Exploration means the drone occasionally tries alternative routes or altitudes it has not used before, even if the currently known route seems fine. This is essential because the environment is dynamic — wind patterns, temporary obstacles (e.g., a crane or another aircraft), and no-fly zone boundaries can change over time. Without exploration, the agent's route knowledge becomes stale and suboptimal.

4. Balancing Techniques and Effect on Route Optimization

Standard strategies apply here as well:

- Epsilon-greedy exploration with a decaying ε: early in training/deployment the drone explores more routes; as confidence grows, it increasingly exploits the best-known path, with occasional exploration retained to adapt to change.
- Softmax/Boltzmann exploration: routes are chosen probabilistically in proportion to their estimated value, so promising but untested routes still get a reasonable chance of being tried.
- Upper Confidence Bound-style bonuses: routes flown less often receive an exploration bonus, ensuring the drone periodically re-evaluates alternatives rather than fixating on one path.
- Safe exploration bounds: exploration is restricted to a set of pre-approved airspace corridors, ensuring that trying a new route never risks flying into a genuinely unsafe or illegal zone.

5. Worked Illustration

Suppose a drone has always taken Route A between the depot and a delivery zone, which is known to be reliable. During a delivery, an exploratory action causes it to test Route B, which follows a slightly longer ground path but flies at an altitude with a strong tailwind that day. The trip is completed faster and with lower battery consumption than usual. This positive outcome updates the drone's value estimate for Route B upward, and over subsequent deliveries under similar wind conditions the fleet's routing policy shifts toward preferring Route B — a discovery that pure exploitation of Route A alone would never have produced.

6. Outcome

By balancing exploration and exploitation, the drone delivery system continuously refines its route library — exploiting fast, safe, energy-efficient paths for routine deliveries while periodically exploring new paths that can adapt to weather changes, new obstacles, or airspace restrictions, resulting in progressively shorter delivery times and lower energy consumption over the fleet's operational life.

7. Conclusion

A well-tuned exploration-exploitation strategy allows a drone delivery fleet to continually adapt its navigation policy to a changing airspace environment while still providing dependable, efficient day-to-day delivery performance, illustrating why this trade-off is one of the most fundamental design decisions in any practical RL system.

8. Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency.

Introduction

In a manufacturing setting, a robotic arm performs repetitive assembly tasks such as picking, placing, and fitting components on a production line. RL can be used to optimize the sequence and precision of movements to reduce cycle time, minimize defects, and reduce wear/energy consumption. Because assembly tasks require fine, continuous motor control rather than a small set of discrete moves, this problem is typically framed as a continuous-control MDP solved with Deep RL rather than classical tabular methods.

1. State, Action and Reward (Framework)

The state includes the current joint angles/position of the robotic arm, the position and orientation of the component to be assembled (often from camera/vision sensors), the current stage of the assembly sequence, and sensor feedback such as force/torque at the gripper. The

action space includes joint movements (or end-effector displacement in x, y, z, and rotation), gripper open/close commands, and speed adjustments. The reward function provides a positive reward for correctly completing an assembly step, a negative reward for misalignment, dropped parts, or exceeding force thresholds (which could damage components), and a small negative reward for time taken, to encourage faster cycles.

2. Policy (π)

The policy defines how the robot maps its current sensed state to a motor action. Initially, an untrained policy may produce jerky, imprecise, or inefficient movements. Through training (commonly using Deep RL algorithms such as PPO, TRPO, or DDPG suited to continuous control), the policy is iteratively refined so that the robot's action selection consistently produces smooth, precise, and fast assembly motions. A stochastic policy is often used during training to promote exploration of slightly different grasping angles and approach trajectories, gradually narrowing toward a near-deterministic, highly reliable policy once performance stabilizes.

3. Value Function and Efficiency Improvement

- The value function estimates the expected long-term reward (successful completions, low error, and low time cost) from a given arm configuration, while the action-value function Q(s,a) evaluates specific motion choices. During training, the value function is updated using observed outcomes, and it captures subtle long-term consequences of an action — for example, a slightly slower but more centered gripper approach may have a higher value than a faster but riskier approach, because it reliably avoids costly misalignment errors later in the sequence.
- As the value function becomes more accurate, the policy is updated to act greedily with respect to it (policy improvement), and this cycle of policy evaluation and policy improvement (generalized policy iteration) progressively converges toward an optimal or near-optimal assembly policy.

4. Worked Illustration

Consider a robotic arm inserting a pin into a tightly toleranced hole. An untrained policy might approach the hole too quickly, occasionally missing alignment and damaging the part (a large negative reward). As training proceeds, the value function learns that states where the gripper is well-centered above the hole before descending have a much higher expected future reward than states where the descent begins slightly off-center, even though the off-center approach might initially seem 'faster.' The improved policy therefore learns to spend a fraction of a second more on fine alignment before insertion, dramatically reducing the defect rate while only marginally increasing cycle time — a net efficiency gain once defect-related rework is accounted for.

5. Outcome

Over many training iterations (in simulation and then fine-tuned on the real robot), the manufacturing robot learns highly efficient motion trajectories that reduce cycle time, minimize defective assemblies, reduce mechanical wear from unnecessary or jerky movements, and adapt automatically to minor variations in component position — resulting in a significant improvement in overall assembly line throughput and quality.

6. Conclusion

By combining a precisely defined state-action-reward structure with the iterative refinement of policy and value function through generalized policy iteration, an RL-driven manufacturing robot can achieve consistently higher precision, speed, and reliability than manually programmed motion sequences, while also being able to adapt automatically to new products or minor process variations without being reprogrammed from scratch.

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance.

Introduction

In online advertising, an RL agent decides which ad to show to which user in a given context, with the goal of maximizing click-through rate (CTR) and, more broadly, long-term advertiser and platform revenue. As with other RL applications, the reward function used to represent 'a good outcome' is difficult to design correctly, and mistakes here directly translate into poor real-world system behavior. The problem is often modelled as a contextual bandit or a full sequential MDP when the platform also cares about a user's engagement across multiple sessions, not just a single ad impression.

1. State, Action and Reward (Context)

The state (context) typically includes the user's demographic and behavioural profile, browsing/search history, the current page or search query context, device type, and time of day. The action is the specific ad (or ranked set of ads) shown to the user from the available inventory. The immediate reward is usually a binary click/no-click signal, though richer reward formulations also incorporate post-click conversion events.

2. Key Reward Design Challenges

The main challenges include:

- Short-term vs long-term objective mismatch: A reward based purely on immediate clicks can encourage the agent to favor clickbait-style ads that generate clicks but lead to poor conversion, poor user experience, or advertiser dissatisfaction — clicks are not the same as true business value.

- Sparse and noisy feedback: Most impressions do not result in a click, so the reward signal is extremely sparse (mostly zero), making it statistically difficult for the agent to learn which factors truly drive clicks.
- Delayed conversion signals: The ultimate value of an ad (a purchase or sign-up) may occur long after the click, so a reward tied only to clicks ignores whether those clicks were valuable.
- User fatigue and long-term engagement: Repeatedly showing the same type of high-CTR ad can annoy users and reduce their engagement with the platform over time; a reward that only measures immediate CTR does not capture this long-term cost.
- Popularity/position bias: Ads shown in more prominent positions naturally get more clicks regardless of relevance, so a naively designed reward can cause the agent to learn a biased notion of what makes an ad 'good.'
- Fairness and diversity: Optimizing purely for CTR can create a feedback loop that shows the same few high-performing ads/advertisers repeatedly, unfairly disadvantaging new or smaller advertisers.

3. Impact of Poor Reward Design on Performance

If these challenges are not properly addressed:

- The system may over-optimize for clickbait, harming advertiser trust and long-term platform revenue even while short-term CTR metrics look good.
- The learned policy can develop a strong bias toward already-popular ads, reducing exploration of potentially better but less-tested ads (an exploration-exploitation failure driven by reward design).
- User experience can degrade over time (ad fatigue, irrelevant ads), leading to lower long-term user retention despite high immediate CTR.
- The agent may appear to perform very well in offline evaluation but underperform when deployed live, because the reward function did not reflect real-world objectives such as conversions and long-term satisfaction.

4. Worked Illustration

Suppose the platform rewards the agent purely for clicks. The agent quickly learns that ads with sensational, exaggerated headlines attract more clicks than accurately described ads, even though users who click the sensational ads rarely convert and often leave the landing page immediately (a high 'bounce rate'). Advertisers, seeing poor return on their spend despite high click volume, begin to lose trust in the platform and reduce their bidding, ultimately harming platform revenue — a long-term negative outcome that a purely click-based reward could never have detected, since it only measured the immediate, superficial signal.

5. Mitigation

Robust systems typically combine multiple signals into the reward (click, conversion, and estimated long-term user satisfaction), use counterfactual/off-policy evaluation to avoid position bias, and incorporate exploration bonuses or diversity constraints to keep the ad ecosystem fair and adaptive.

6. Conclusion

A robust online advertising RL system must move beyond a naive click-only reward toward a carefully engineered, multi-signal reward that reflects true business value, long-term user satisfaction, and fairness, since even small flaws in reward design can compound over millions of impressions into significant, hard-to-reverse damage to user trust and platform revenue.

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function.

Introduction

A smart irrigation system uses RL to decide when and how much to irrigate different zones of a field, with the objective of maximizing crop yield/health while minimizing water usage and cost. This is an important real-world application because water is a scarce resource, and the environment (soil, weather) is highly dynamic. Because soil moisture today directly affects the irrigation need of tomorrow, this is naturally modelled as a multi-step Markov Decision Process rather than a series of independent, one-off decisions.

1. State Space (S)

The state should represent everything relevant to the irrigation decision:

- Current soil moisture level in each field zone (from soil sensors).
- Weather conditions and short-term forecast (temperature, humidity, expected rainfall).
- Crop growth stage (since water requirements differ across growth stages).
- Time since last irrigation and amount of water used recently.
- Evapotranspiration rate (how quickly water is being lost from soil and plants).

2. Action Space (A)

The possible irrigation actions include:

- Irrigate a specific zone with a specific volume of water (or turn a specific valve/sprinkler on for a set duration).
- Do not irrigate (hold) if soil moisture is already sufficient or rainfall is expected soon.

- Adjust irrigation intensity (e.g., light, medium, heavy) based on current need.

3. Reward Function (R)

The reward mechanism should balance crop health against water conservation:

- A positive reward for keeping soil moisture within the optimal range for the current crop growth stage.
- A negative reward proportional to the amount of water used, to discourage unnecessary irrigation.
- A large negative reward for under-watering that leads to crop stress (moisture falling below a critical threshold) or over-watering that causes waterlogging/root damage.
- A positive reward correlated with eventual crop yield/health indicators (if available from sensors or historical data), tying short-term irrigation decisions to long-term agricultural outcomes.

4. Algorithmic Approach

For a discretized representation of soil moisture levels and a small set of irrigation intensities, tabular Q-Learning or Dynamic-Programming-based Value Iteration can be applied directly, since the field can be modelled as a finite MDP with a manageable number of states per zone. For finer-grained continuous control of water volume, algorithms such as DDPG or PPO are more appropriate. Because agricultural seasons are long and real-world experimentation is costly (a poor policy could damage an actual crop), the agent is typically trained extensively in a simulated agronomic model calibrated with historical soil and weather data before any field deployment.

5. Worked Illustration

Consider a field zone where the weather forecast predicts rainfall within the next six hours. A naive fixed-schedule irrigation system would still irrigate on schedule, wasting water and potentially causing waterlogging once the rain arrives. The RL agent, having learned to incorporate forecast information into its state, instead withholds irrigation for that zone, receiving a positive reward both from water conservation and from the natural rainfall keeping soil moisture within target range. Over an entire growing season, thousands of such small, forecast-aware decisions accumulate into substantial water savings.

6. Learning and Outcome

Using an RL algorithm such as Q-Learning (for a discretized state-action space) or DDPG/PPO (for continuous water-volume control), the agent learns, through repeated seasonal cycles (often first in a simulated agronomic model), a policy that irrigates just enough, just in time — for example, learning to skip irrigation when rain is forecast, or increasing irrigation slightly during a crop's critical growth phase. Over time this results in significant water savings compared to

fixed-schedule irrigation, while maintaining or improving crop yield, demonstrating the value of RL-based precision agriculture.

7. Conclusion

By explicitly modelling soil, weather, and crop-stage information in the state space and carefully balancing water conservation against crop-health penalties in the reward function, an RL-based smart irrigation system can substantially reduce water consumption while sustaining or improving yield, making it a practically valuable and environmentally significant application of reinforcement learning in precision agriculture.

RUBRICS FOR CALCULATION

Criteria	Weig htage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understand ing with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretati on of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understan dable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand